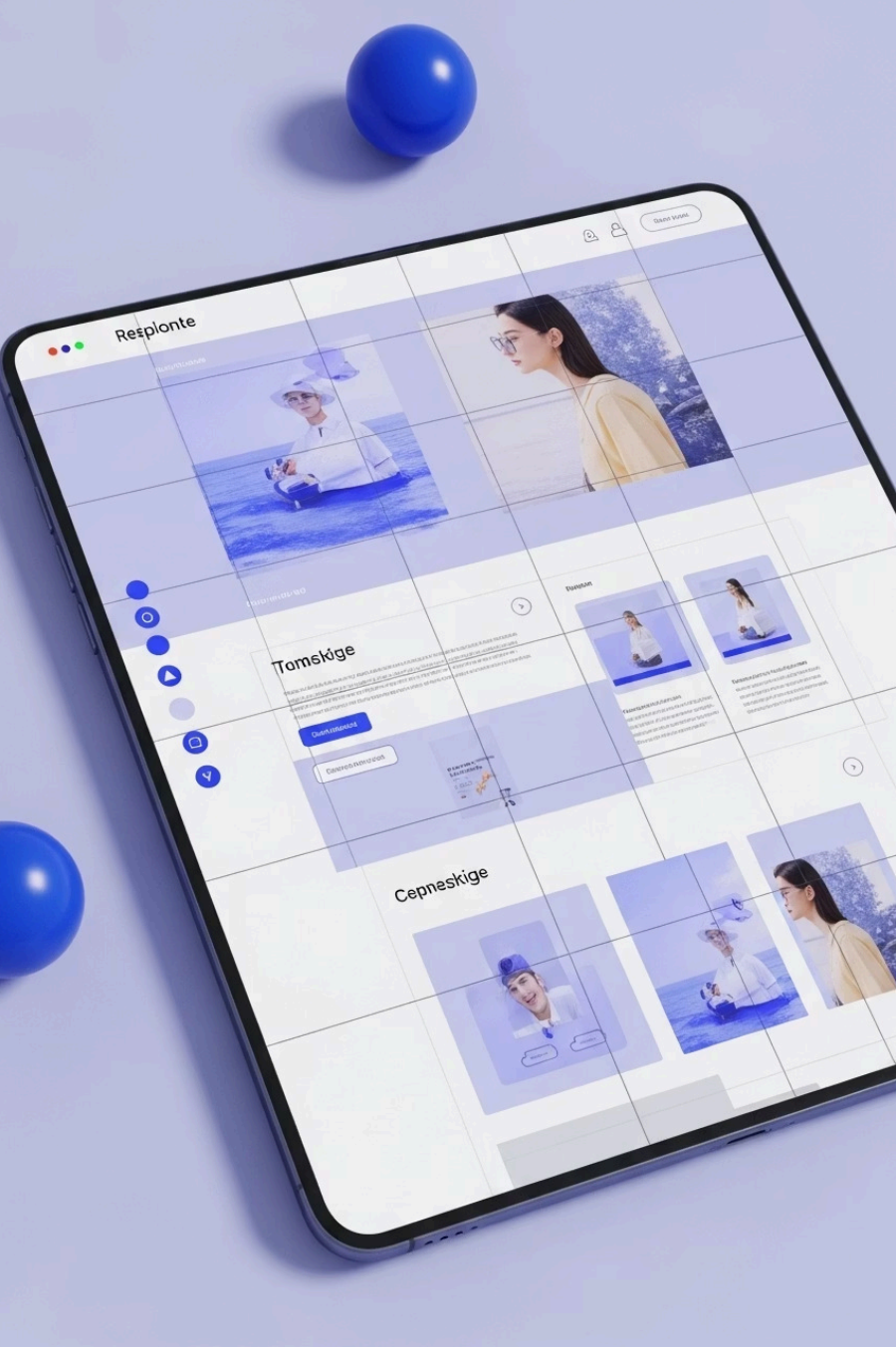




Dominando CSS Grid: La Revolución del Diseño Web

Una guía completa para desarrolladores y estudiantes que quieren dominar el sistema de diseño bidimensional más potente de CSS.

¿Qué es CSS Grid?

CSS Grid es un sistema de diseño **bidimensional** que permite organizar elementos en filas y columnas simultáneamente, facilitando la creación de:

- Layouts complejos con control preciso
- Diseños responsive sin media queries
- Estructuras web que antes requerían soluciones complicadas

A diferencia de Flexbox (unidimensional), Grid ofrece control sobre ambas dimensiones a la vez.



Primeros Pasos: Activando Grid

Sintaxis Básica

HTML

```
<div class="container">

  <div class="item_grid">Div 1</div>

  <div class="item_grid">Div 2</div>

  <div class="item_grid">Div 3</div>

  <div class="item_grid">Div 4</div>

  <div class="item_grid">Div 5</div>

  <div class="item_grid">Div 6</div>

</div>
```

CSS

```
.container{
  background-color: ghostwhite;
  display: grid;
}

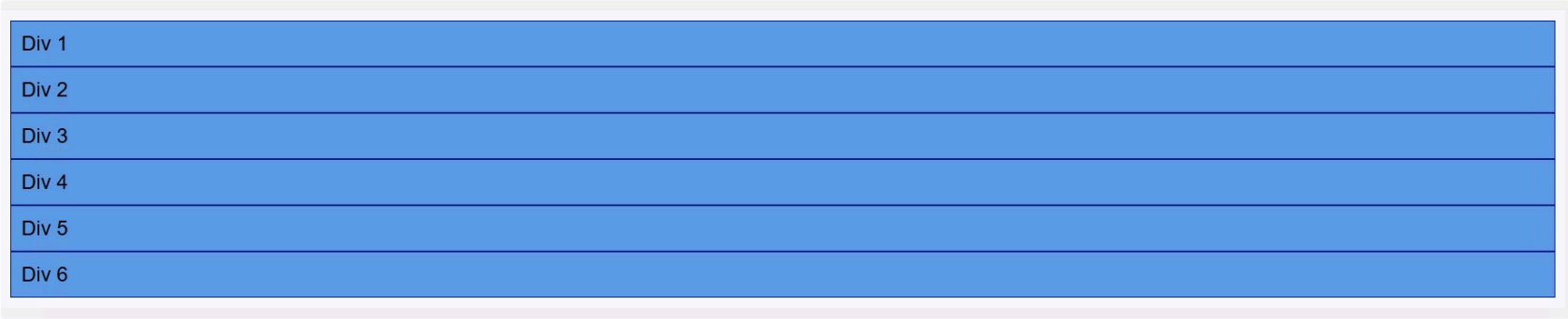
.item_grid, .container{
  padding: .5rem;
}

.item_grid{
  background-color: cornflowerblue;
  border: 1px solid darkblue;
}
```

Al activar grid, inicialmente no verás cambios evidentes. Los elementos se apilarán en una sola columna hasta que definas la estructura de filas y columnas.

El contenedor ahora está preparado para recibir las propiedades específicas de Grid que determinarán su estructura.

Ejemplo:



Es importante destacar que `display: grid` se aplica únicamente al contenedor padre, mientras que sus hijos se convierten automáticamente en elementos grid.

Definiendo la Estructura del Grid

HTML

```
<div class="container">

<div class="item_grid">Div 1</div>

<div class="item_grid">Div 2</div>

<div class="item_grid">Div 3</div>

<div class="item_grid">Div 4</div>

<div class="item_grid">Div 5</div>

<div class="item_grid">Div 6</div>

</div>
```

Define el número de columnas y sus tamaños. En este ejemplo: primera columna de 100px, segunda de 200px, y tercera que ocupa el espacio restante (auto).

CSS(grid-template-row | grid-template-columns)

```
.container{
  background-color: ghostwhite;
  display: grid;
  grid-template-columns: 100px 200px 100px; /* 3 values = 3
columns */
  grid-template-rows: 4rem 2rem; /* 2 values = 2 rows */
}

.item_grid, .container{
  padding: .5rem;
}

.item_grid {
  background-color: cornflowerblue;
  border: 1px solid darkblue;
}
```

Define el número de filas y sus tamaños. Aquí: primera fila de 150px y segunda que ocupa el espacio restante.

Ejemplo:

Div 1	Div 2	Div 3
Div 4	Div 5	Div 6

La Unidad fr: Fracciones Flexibles

La unidad `fr` (fracción) es exclusiva de CSS Grid y representa una parte del espacio disponible en el contenedor.

```
.container{
  background-color: ghostwhite;
  display: grid;
  grid-template-columns: 1fr 2fr 1fr;
}
.item_grid, .container{
  padding: .5rem;
}
.item_grid {
  background-color: cornflowerblue;
  border: 1px solid darkblue;
}
```

En este ejemplo, el espacio horizontal se divide en 4 partes (1+2+1). La columna central ocupa 2/4 partes (50%), mientras que las columnas laterales ocupan 1/4 parte (25%) cada una.

Div 1	Div 2	Div 3
Div 4	Div 5	Div 6

```
<div class="container">

<div class="item_grid">Div 1</div>

<div class="item_grid">Div 2</div>

<div class="item_grid">Div 3</div>

<div class="item_grid">Div 4</div>

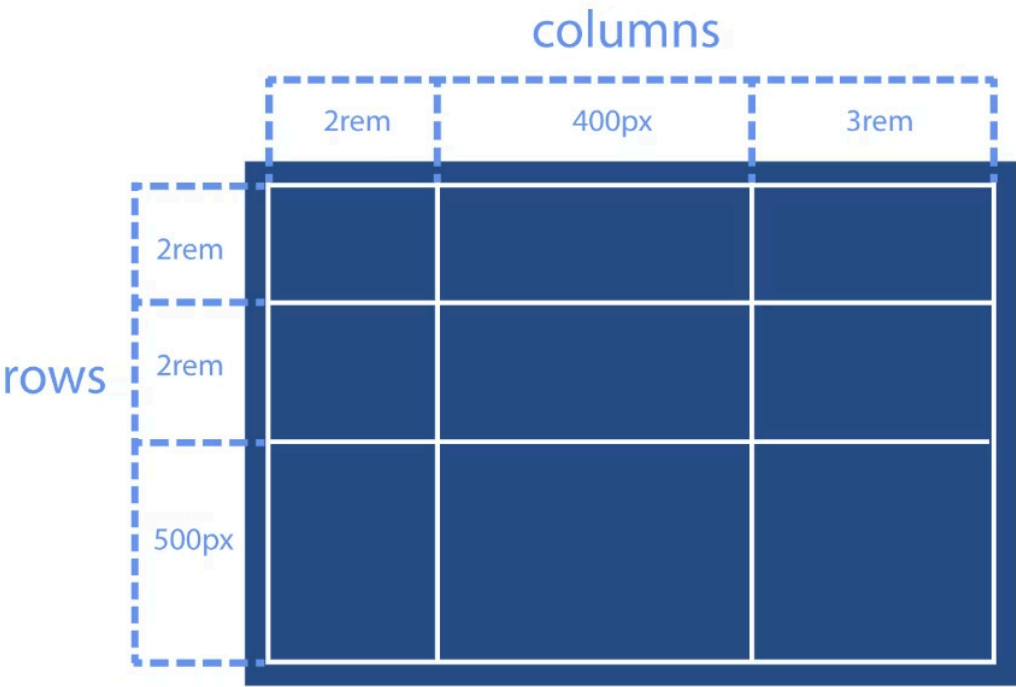
<div class="item_grid">Div 5</div>

<div class="item_grid">Div 6</div>

</div>
```

La unidad `fr` es perfecta para diseños responsive, ya que se adapta automáticamente al tamaño del contenedor sin necesidad de media queries.

```
grid-template-columns: 2rem 400px 3rem;
grid-template-rows: 2rem 2rem 500px;
```



Función repeat(): Patrones Repetitivos

Sintaxis y Uso

```
.contenedor {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  grid-template-rows: repeat(2, 100px);  
  border: 2px solid #403ccf;  
}
```

La función `repeat()` permite definir patrones repetitivos, evitando código redundante. El primer parámetro indica cuántas veces se repite, y el segundo qué patrón se repite.

Patrones Complejos

```
.contenedor {  
  display: grid;  
  grid-template-columns:  
    repeat(2, 100px)  
    repeat(3, 1fr);  
  border: 2px solid #403ccf;  
}
```

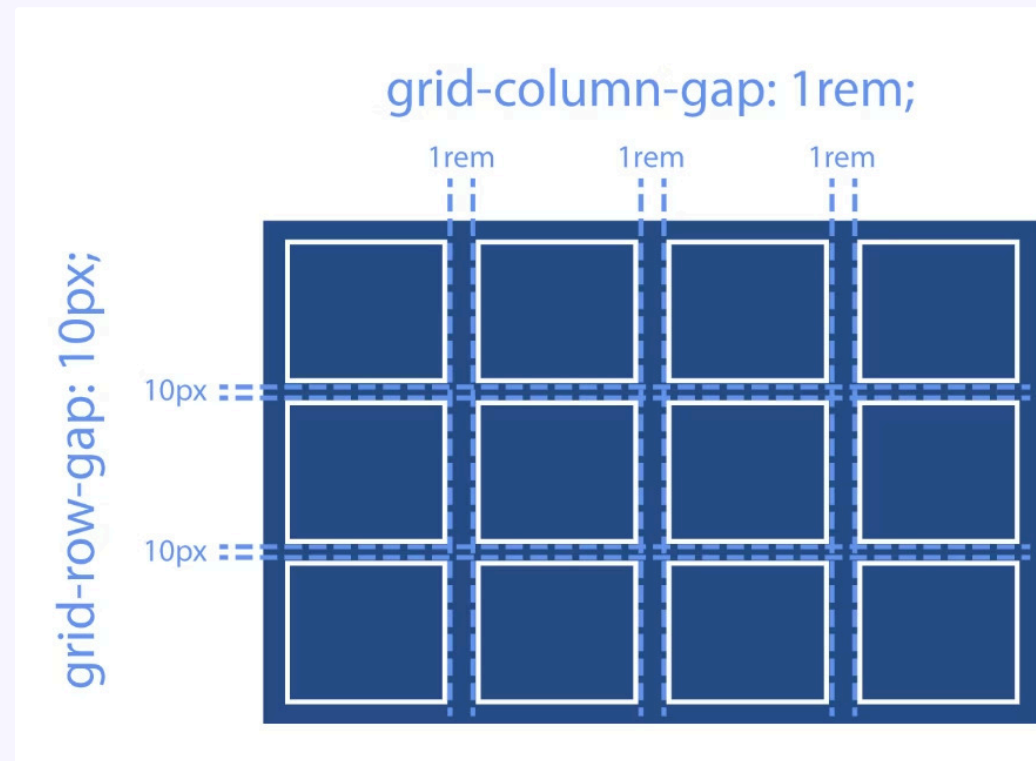
También puedes combinar la función `repeat()` con otros valores para crear estructuras más complejas. En este ejemplo: 2 columnas de 100px seguidas de 3 columnas flexibles de igual tamaño.

Espaciado entre Elementos con gap

```
.contenedor {  
  display: grid;  
  grid-template-columns: repeat(4, 1fr);  
  grid-template-rows: repeat(3, 1fr);  
  grid-column-gap: 1rem;  
  grid-row-gap: 10px;  
  /*usando solo gap: aplica la misma medida a ambos*/  
  border: 2px solid #403ccf;  
}
```

La propiedad `gap` define el espacio entre filas y columnas del grid. Es una forma más limpia de crear márgenes entre elementos sin afectar los bordes externos del contenedor.

También puedes usar `row-gap` y `column-gap` para controlar el espaciado horizontal y vertical por separado.



Nota: Anteriormente se usaba `grid-gap`, pero ahora `gap` es la propiedad estándar y funciona tanto en Grid como en Flexbox.

Alineación Horizontal con `justify-items`

La propiedad `justify-items` define cómo se alinean los elementos del grid dentro de sus celdas a lo largo del **eje horizontal** (en la dirección de la fila), cuando el contenido del elemento no ocupa todo el ancho disponible de la celda.

1 **stretch** (valor por defecto)
El elemento se expande para ocupar todo el ancho de la celda de la cuadrícula.

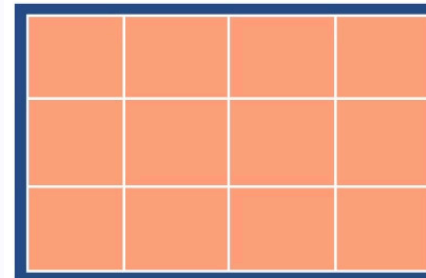
2 **start**
El elemento se alinea al inicio (izquierda) de la celda.

3 **end**
El elemento se alinea al final (derecha) de la celda.

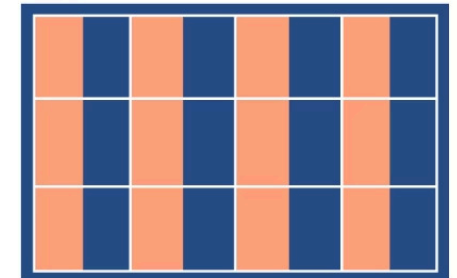
4 **center**
El elemento se centra horizontalmente dentro de la celda.

```
.parent {  
  background-color: ghostwhite;  
  display: grid;  
  grid-column-gap: .5rem;  
  grid-row-gap: 1rem;  
  grid-template-columns: repeat( 3, 1fr );  
  grid-template-rows: repeat( 3, 5rem );  
  justify-items: center;  
}
```

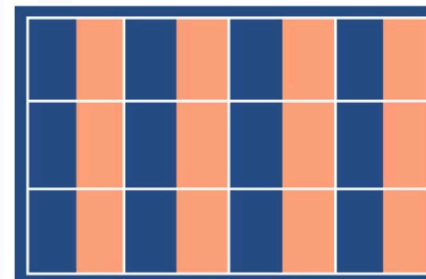
stretch



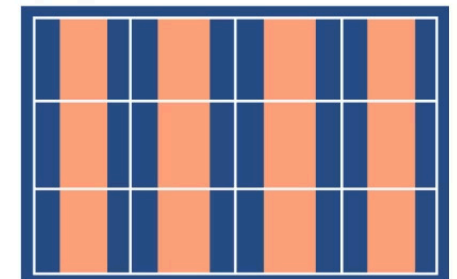
start



end



center



Esta propiedad se aplica al **contenedor grid** y afecta a todos sus elementos hijos.

Alineación Vertical con `align-items`

La propiedad `align-items` controla cómo se alinean los elementos hijos del grid a lo largo del **eje vertical** (en la dirección de la columna) dentro de sus celdas, cuando el contenido del elemento no ocupa toda la altura disponible de la celda.

1 `stretch` (valor por defecto)
El elemento se estira para ocupar toda la altura disponible de su celda.

2 `start`
El elemento se alinea al inicio (arriba) de la celda, ocupando solo la altura necesaria para su contenido.

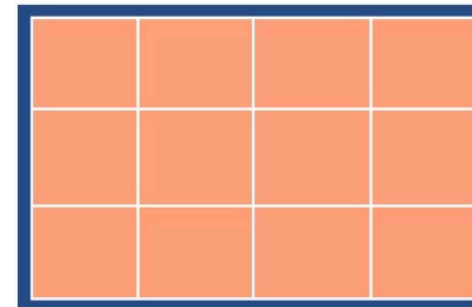
3 `end`
El elemento se alinea al final (abajo) de la celda, ocupando solo la altura necesaria para su contenido.

4 `center`
El elemento se centra verticalmente dentro de la celda, ocupando solo la altura necesaria para su contenido.

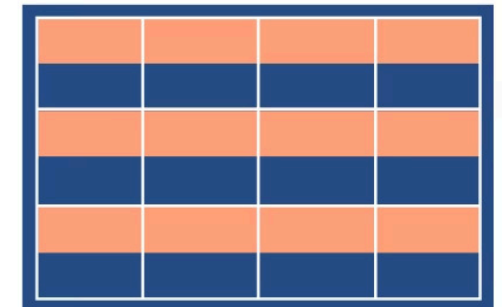
Esta propiedad se aplica al **contenedor grid** y afecta a la alineación de todos sus elementos hijos.

```
.parent {  
  background-color: ghostwhite;  
  display: grid;  
  grid-column-gap: .5rem;  
  grid-row-gap: 1rem;  
  grid-template-columns: repeat( 3, 1fr );  
  grid-template-rows: repeat( 3, 5rem );  
  align-items: center;  
}
```

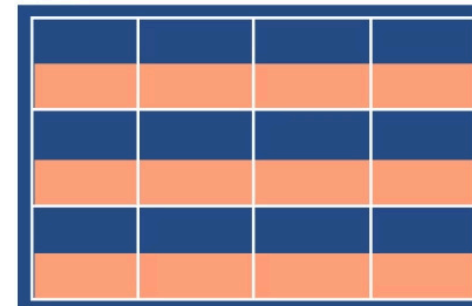
`stretch`



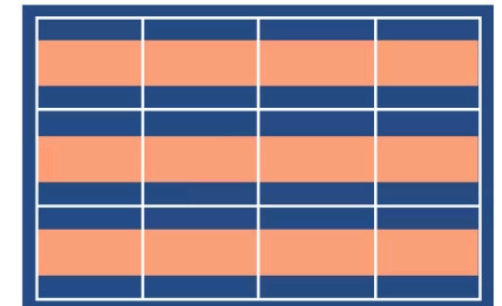
`start`



`end`



`center`



Esta propiedad se aplica al **contenedor grid** y afecta a todos sus elementos hijos.

Alineación con place-items

Alineación Horizontal y Vertical

```
.contenedor {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  grid-template-rows: repeat(2, 150px);  
  place-items: center;  
  gap: 10px;  
  border: 2px solid #403ccf;  
}
```

place-items es un atajo que combina justify-items (alineación horizontal) y align-items (alineación vertical) en una sola propiedad.

Valores Posibles

- start: Alinea al inicio de la celda
- end: Alinea al final de la celda
- center: Alinea al centro de la celda
- stretch: Estira para ocupar toda la celda (valor por defecto)

Si especificas un solo valor, se aplica tanto horizontal como verticalmente. Si usas dos valores, el primero es para la alineación vertical y el segundo para la horizontal.

Posicionamiento con grid-row-start, grid-row-end, grid-column-start y grid-column-end

```
.container{
  background-color: ghostwhite;
  display: grid;
  grid-column-gap: .5rem;
  grid-row-gap: 1rem;
  grid-template-columns: repeat( 3, 1fr );
  grid-template-rows: repeat( 3, 5rem );
}
div {
  padding: .5rem;
}
div div {
  background-color: cornflowerblue;
  border: 1px solid darkblue;
}
.div-1 {
  grid-row-start: 1;
  grid-row-end: 3;
}
.div-2 {
  grid-column-start: 2;
  grid-column-end: 4;
}
```

Las propiedades **grid-row-start**, **grid-row-end**, **grid-column-start** y **grid-column-end** permite posicionar un elemento específicamente en el grid utilizando la sintaxis:

propiedad: filaInicio / columnaInicio / filaFin / columnaFin;

```
<div class="container">

  <div class="div-1">Div 1</div>

  <div class="div-2">Div 2</div>

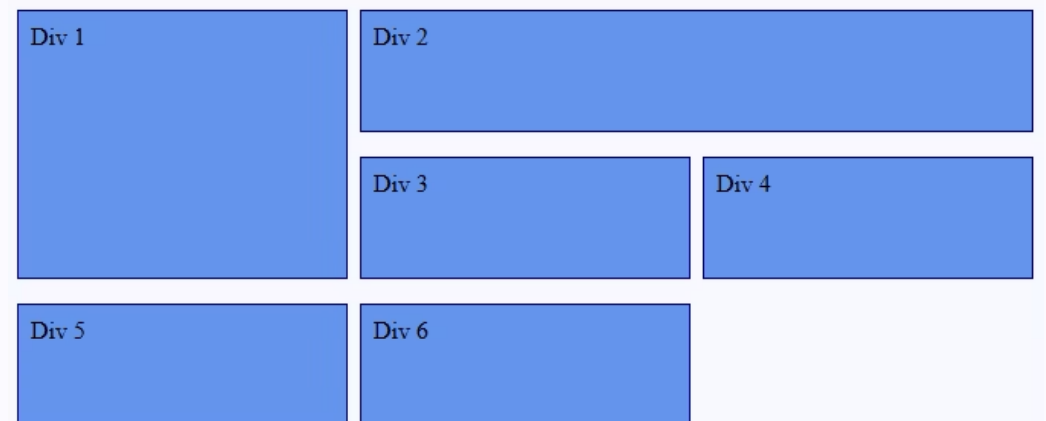
  <div>Div 3</div>

  <div>Div 4</div>

  <div>Div 5</div>

  <div>Div 6</div>

</div>
```



En el ejemplo, `.div-1` ocupa dos filas y una columna (desde la fila 1 hasta la 3, sin incluir la 3).

En el ejemplo, `.div-2` ocupa la columna 2 hasta la 4 usando solo una fila.

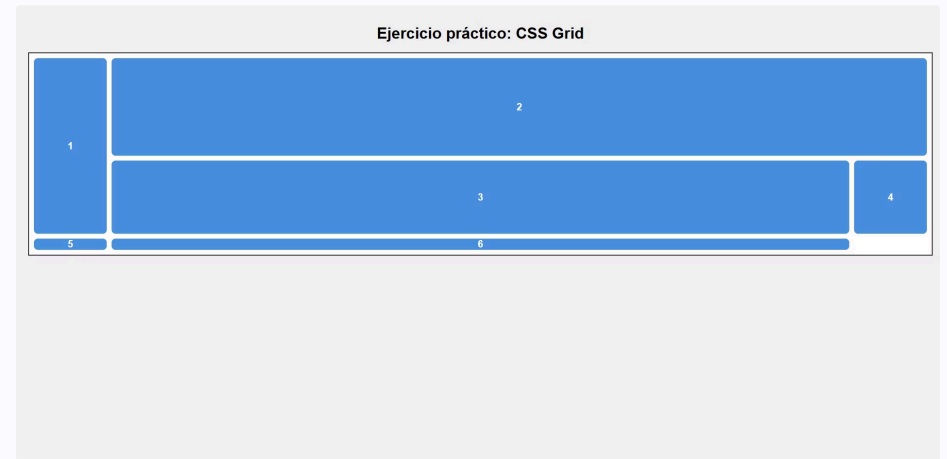
Los números hacen referencia a las **líneas del grid**, no a las celdas. La primera línea es 1, la segunda es 2, etc.

Ejercicio Práctico: Construyendo un Grid Completo

Ahora que hemos cubierto los conceptos fundamentales de CSS Grid, es hora de poner en práctica lo aprendido. Este ejercicio te desafiará a combinar varias propiedades para crear un diseño de cuadrícula específico.

Consigna del Ejercicio

1. Crea un contenedor que use CSS Grid con **3 columnas (150px, 1fr, 150px)** y **3 filas(200px, 150px auto)**.
2. Coloca **6 cajas (div)** dentro del grid.
3. Alinea todos los ítems con `align-items` y `justify-items` para centrarlos en sus celdas(`align-items: center, justify-content: center;`).
4. Usa posicionamiento manual en algunos elementos:
 - Que el primer `div` ocupe **dos filas de alto**.
 - Que el segundo `div` ocupe **dos columnas de ancho**.
5. Explica todo en **comentarios** dentro del código.

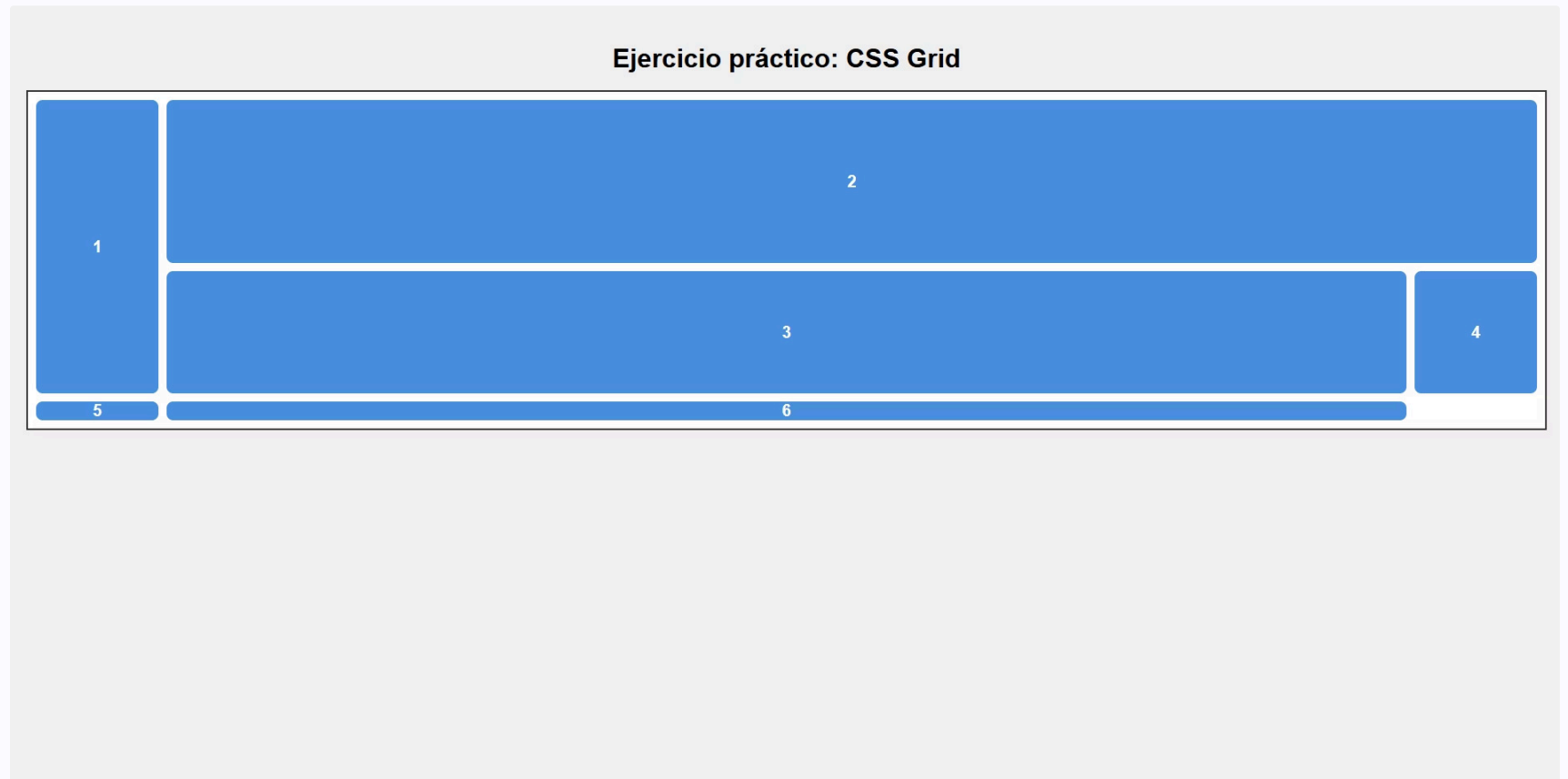


El objetivo es recrear un diseño similar al de la imagen, demostrando tu comprensión de la alineación y el posicionamiento manual de elementos en CSS Grid.

¡Tómate tu tiempo para experimentar con las propiedades y ver cómo cada una afecta el diseño final!

¡Tu Css en Acción!

Ahora que has guardado y abierto tu archivo HTML y CSS, deberías ver tu página web renderizada en el navegador con todos los elementos de formato aplicados, tal como los has definido en el código. ¡El resultado debería ser visualmente impactante y bien estructurado!



✅ ¡Felicidades! Has completado con éxito esta sección. Tu pagina se ve genial y estás un paso más cerca de dominar el diseño web.